

SYSTEM SPECIFICATION

QuestVector

Local-first career navigation & job-matching platform — Python desktop implementation

Document version: 1.0.0 (tracks `questvector.__version__`)

Covers: the Python desktop build only — see §10 for the Web Cockpit's status

Maintainer: Dusten Harrison

Prepared: September 2026

142 automated tests, all passing	0 mypy / pyflakes findings	4 interchangeable LLM providers	1 of 2 designed surfaces built (CLI + desktop)
--	--	---	--

Contents

1. Overview & Scope	
2. System Architecture	
3. Distribution & Security Model	
4. Core Domain Model	
5. Matching Engine — the G-Force Score	
6. Mission Template Schema	
7. CLI Reference (<code>qv</code>)	
8. Desktop Application	
9. LLM Narrative-Generation Subsystem	
10. Storage, File Layout & Repository Structure	
11. Quality, Testing & Packaging	
12. Roadmap — Web Cockpit (Not Yet Built)	

1 Overview & Scope

QuestVector is an open-source, local-first career management tool for experienced IT professionals. A user maintains their career history as three plain Markdown files (capabilities, chronology, and quantified impact); QuestVector scores that history against a target job description using a set of weighted, role-specific "mission templates," surfaces coverage gaps and red flags, and can compile the result into a tailored PDF or DOCX resume bundle — optionally with an AI-drafted narrative summary.

The project was originally specified as a browser application (React/Tailwind UI over a Rust-compiled WebAssembly core). That specification is referenced throughout this document where it explains a design choice, but **no browser/Wasm code has been written**. Everything described as implemented in this document is the Python reimplementation: a portable core engine, a stdlib-based CLI (`qv`), and a native desktop shell.

Aspect	Python Desktop (this build)	Web Cockpit
Status	Implemented, tested, runs today	Design target only — no code exists
Stack	Pure Python core + <code>qv</code> CLI + pywebview shell	React/Tailwind UI + Rust-compiled Wasm core (planned)
Distribution	Clone + <code>pip install</code> , run locally	Would clone & run locally when built — never hosted
Storage	Local filesystem + AES-256-GCM encrypted vault file	File System Access API + IndexedDB (planned)

READING THIS DOCUMENT

Sections 2–9 and 11 describe the Python build exactly as it exists in the repository today, verified against the source and the passing test suite. Section 12 is explicitly forward-looking and is marked as such throughout — nothing in it should be read as already built.

2 System Architecture

Two entry points — the `qv` CLI and the pywebview desktop shell — sit on top of one shared core engine. Both entry points talk to the core through the same plain Python function calls; neither has any logic of its own beyond argument parsing (CLI) or JS-bridge marshalling (desktop). All persistent state is local files under a user-chosen workspace directory. The one path that leaves the local machine is optional, explicit LLM narrative generation.

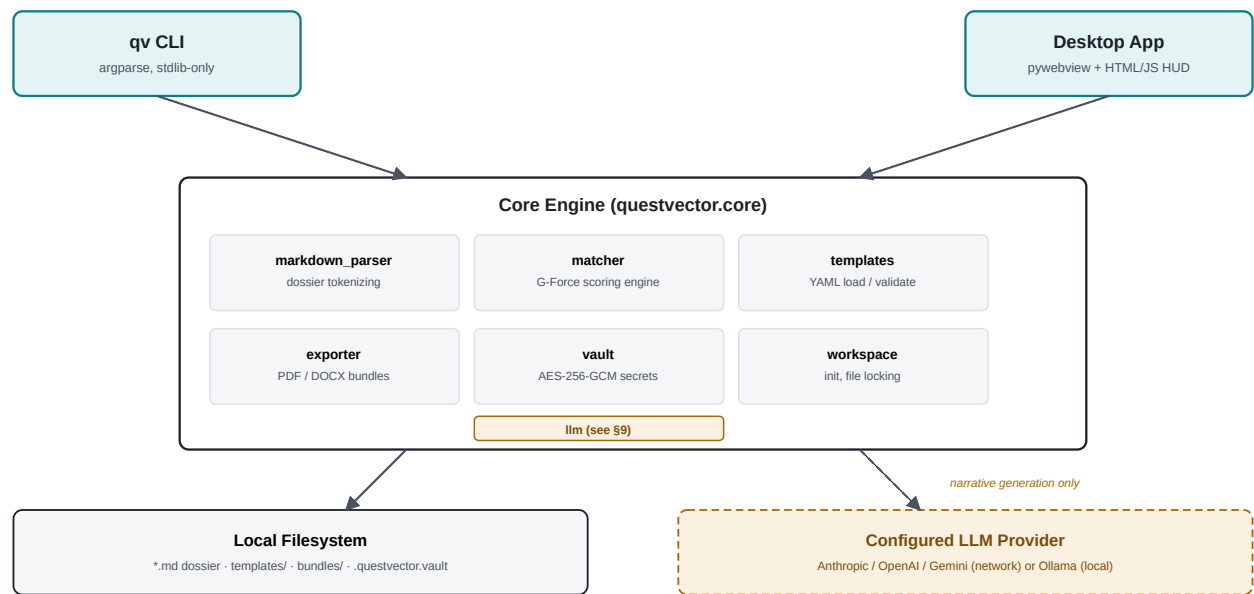


Fig. 1 — Both entry points call the same core engine; only the LLM subsystem talks to anything off-machine, and only when explicitly invoked.

2.1 Component responsibilities

Module	Responsibility
<code>core.markdown_parser</code>	Parses the three dossier Markdown files into structured, tokenized data; tokenizes free-text job descriptions the same way.
<code>core.matcher</code>	Runs the Micro-Diff Matching Engine described in §5 — the only place scoring logic lives.
<code>core.templates</code>	Loads, validates, and (with <code>--fix</code>) auto-corrects mission template YAML files.
<code>core.exporter</code>	Compiles a dossier plus an optional match summary and narrative into a PDF (reportlab) or DOCX (python-docx) bundle.
<code>core.vault</code>	Encrypts/decrypts the local secrets file (§3, §9).
<code>core.workspace</code>	Initializes a workspace directory and provides advisory cross-process file locking.
<code>core.llm</code>	The narrative-generation provider abstraction — see §9.
<code>cli</code>	The <code>qv</code> argparse command surface — thin orchestration over the modules above.
<code>desktop.app</code>	The pywebview <code>Api</code> class exposed to the JS front end; every method returns a JSON-safe <code>{"ok": ...}</code> envelope, never an unhandled exception.

3 Distribution & Security Model

DISTRIBUTION POLICY

Nothing about this project is published externally. No hosted domain, no npm-registry package, no telemetry endpoint — for either implementation, ever. The project is cloned and run entirely from local files.

The one deliberate exception is described fully in §9: LLM-drafted narrative text is an explicit, user-triggered action, and three of its four supported providers make an outbound call to that provider's own API at the moment it's invoked. This is a runtime network call, not a distribution mechanism — it doesn't change how the software itself is obtained or run.

3.1 Encrypted local vault

Secrets (LLM provider selection, per-provider API keys, model/host overrides) are stored in a single file per workspace (`.questvector.vault`), encrypted at rest:

Parameter	Value
Cipher	AES-256-GCM (authenticated encryption)
Key derivation	PBKDF2-HMAC-SHA256, 390,000 iterations
Salt / nonce	16 / 12 random bytes, generated per save
File permissions	Restricted to owner read/write on POSIX systems (best-effort)

A wrong passphrase or corrupted ciphertext fails the GCM authentication tag check and raises a handled `VaultError` — it does not silently return garbage plaintext.

4 Core Domain Model

All domain objects are immutable, slotted dataclasses (`@dataclass(frozen=True, slots=True)`), produced once by a parser or loader and read by downstream consumers — safe to share across the CLI, the desktop bridge, and tests without defensive copying.

Type	Key fields	Produced by
Dossier	capabilities, chronology, impact (tuples)	markdown_parser.parse_dossier
Capability / ChronologyEntry / ImpactStatement	domain / heading, text / bullets, keywords	markdown_parser
JobDescription	source, raw_text, keywords	matcher.parse_job_description
MissionTemplate	template_id, title, categories, red_flag_phrases, gap_coverage_threshold, tone_style, target_page_budget	templates.load_template
TemplateCategory	name, weight, keywords	part of MissionTemplate
MatchResult	g_force_score, category_scores, gaps, red_flags, weak_categories, alert_level	matcher.run_match
AlertLevel	SUCCESS / WARNING / CRITICAL (string enum)	derived by run_match

Every application-raised error derives from a single `QuestVectorError` base (subclasses: `ParseError`, `ValidationError`, `VaultError`, `WorkspaceError`, `ExportError`, `MatchEngineError`, `LLMError`), so callers can catch broadly at a boundary or narrowly where they need to — no bare `except:` anywhere in the codebase.

5 Matching Engine — the G-Force Score

The original specification names this component (the "Micro-Diff Matching Engine") and its CLI surface but never publishes a scoring formula. The algorithm below is this implementation's own, documented design — a transparent, tunable heuristic rather than semantic/NLP matching, so its behavior stays fully predictable and testable.

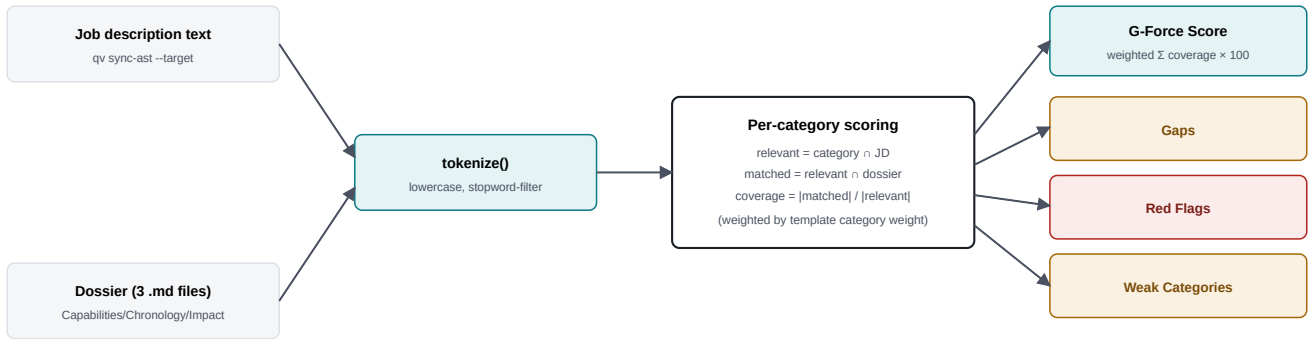


Fig. 2 — The matching pipeline: two independent token sets (JD, dossier) feed per-category scoring against the active mission template.

5.1 Rules

- **Coverage:** for each template category, `relevant` keywords are the category's own keywords that also appear in the JD (falling back to *all* JD keywords if the category declares none). `coverage = matched / relevant`, or `1.0` when nothing in that category was actually asked for — an unmentioned category never penalizes the score.
- **Overall score:** the weight-normalized sum of `category.weight × coverage` across all categories, as a percentage — normalized defensively even if a template's weights don't sum to exactly 1.00.
- **Red flag (coverage-based):** a category is flagged when its weight is ≥ 0.20 and its coverage is exactly zero — a heavily-weighted requirement the dossier says nothing about.
- **Red flag (phrase-based):** any of a template's declared `red_flag_phrases` found verbatim (case-insensitive) in the JD text is flagged independently of dossier coverage.
- **Gaps:** any keyword in a category with coverage < 1.0 , deduplicated across categories.
- **Weak categories:** when a template sets `gap_coverage_threshold`, any category whose coverage falls below it is reported separately — a category can be "weak" overall without any single keyword being individually missing.
- **Alert level:** `CRITICAL` if any red flag fired or score < 40 ; else `WARNING` if there are gaps, weak categories, or score < 70 ; else `SUCCESS`.

6 Mission Template Schema

Mission templates are YAML files, conventionally named `<id>.qv-mission.yaml`. The schema reconciles this project's own category-weighting design with a separately-published community contribution guide, rather than picking one over the other:

```
schema_version: "1.0"
mission_id: devops-senior-mgr      # optional; defaults to the filename stem
metadata:
  title: Senior DevOps Manager     # optional; defaults to mission_id
  author: Jane Contributor         # optional
  version: "1.0.0"                 # optional
  description: A short summary.    # optional
  tags: [devops, platform]        # optional

categories:                        # required – at least one
- name: cloud_infrastructure
  weight: 0.35                     # all weights should sum to 1.00
  keywords: [aws, terraform]
- name: leadership
  weight: 0.25
  keywords: [mentoring, budget]

jd_analysis:                       # optional
  gap_coverage_threshold: 0.5
  red_flag_phrases: [unrealistic requirement, vague job scope]

output_formatting:                # optional
  target_page_budget: 2
  tone_style: executive-tactical
```

Only `name` and `weight` are required per category. A flat legacy `id / title` root is still accepted for backward compatibility when `mission_id / metadata` are absent. Seven starter templates ship with the package (DevOps Senior Manager, Staff Platform Engineer, Cybersecurity Ops Director, Senior IT Project Manager, Senior Systems Administrator, Technical Product Lead, Senior Data Engineer); contributed templates live in a workspace's `templates/community/` directory, separate from the built-in `templates/starter/`.

7 CLI Reference (qv)

Built entirely on the standard library's `argparse` — no third-party CLI framework — per this project's stdlib-first standard.

Command	Key flags	Description
<code>qv launch</code>	<code>--dir</code> , <code>--json</code>	Initialize a workspace: dossier stub files, starter templates, an empty community-templates folder, and a bundles directory. Never overwrites existing files.
<code>qv sync-ast</code>	<code>--target</code> , <code>--template</code> , <code>--dir</code> , <code>-json</code>	Parse a job description, run the matching engine against the workspace dossier, print the G-Force score, gaps, and red flags.
<code>qv export</code>	<code>--bundle</code> , <code>--format {pdf,docx}</code> , <code>--target</code> , <code>--template</code> , <code>--narrative</code>	Compile the dossier (optionally with a match summary and an LLM-generated narrative) into a bundle under <code>bundles/</code> .
<code>qv template validate</code>	<code>path</code> , <code>--fix</code> , <code>--json</code>	Lint a mission template YAML file; with <code>--fix</code> , rescale category weights to sum to 1.00 and persist the correction.
<code>qv vault set-key</code>	<code>--provider</code> , <code>--model</code> , <code>--host</code> , <code>-dir</code>	Store an LLM provider selection (and, for cloud providers, an API key) in the encrypted vault. Prints that provider's setup link and any relevant caveat before prompting.
<code>qv --version</code>	—	Print the installed version, sourced from <code>questvector.__version__</code> (the same value <code>pyproject.toml</code> reads dynamically for packaging metadata).

8 Desktop Application

A native window built with [pywebview](#), themed to the project's "Tactical Ops Flight Deck" HUD visual design. The window hosts a plain HTML/CSS/JS front end (no build step, no framework) across six tabs:

Tab	Purpose
Workspace	Point the app at a local career-workspace directory; launches/initializes it.
Dossier	Read-only view of parsed capabilities, chronology, and impact statements.
Sync-AST	Paste a job description, pick a mission template, run a match — renders the live G-Force gauge, category coverage, gaps, and red flags.
Templates	Validate a mission template YAML file, with an optional auto-fix.
Export	Compile the last scan's dossier + match summary into a PDF or DOCX bundle.
Vault	Choose an LLM provider (with live setup-link/note per selection), store its key, and trigger narrative generation.

8.1 JS ↔ Python bridge contract

Every method on the exposed `Api` class returns a JSON-serializable envelope — `{"ok": true, "data": ...}` or `{"ok": false, "error": "..."}` — deliberately never letting a `QuestVectorError` propagate as an unhandled JS-side exception. `Api` methods are plain, dependency-injectable Python and are exercised directly in tests without ever creating a real window, display server, or GUI toolkit.

9 LLM Narrative-Generation Subsystem

An explicit, user-triggered action (`qv export --narrative` , or the desktop UI's "Generate Narrative" button) that drafts a tailored resume-summary paragraph from dossier excerpts and the target job description. This is the only feature in the build that makes an outbound network call — and, for one of its four provider options, not even that.

Provider	Runs	API key?	Setup
<code>anthropic</code> (Claude)	Remote	Yes	<code>console.anthropic.com</code>
<code>openai</code>	Remote	Yes	<code>platform.openai.com</code> — a separate account/billing relationship from a ChatGPT subscription
<code>gemini</code> (Google)	Remote	Yes	<code>aistudio.google.com</code>
<code>ollama</code>	Local	No	<code>ollama.com</code> — install, run, <code>ollama pull <model></code>

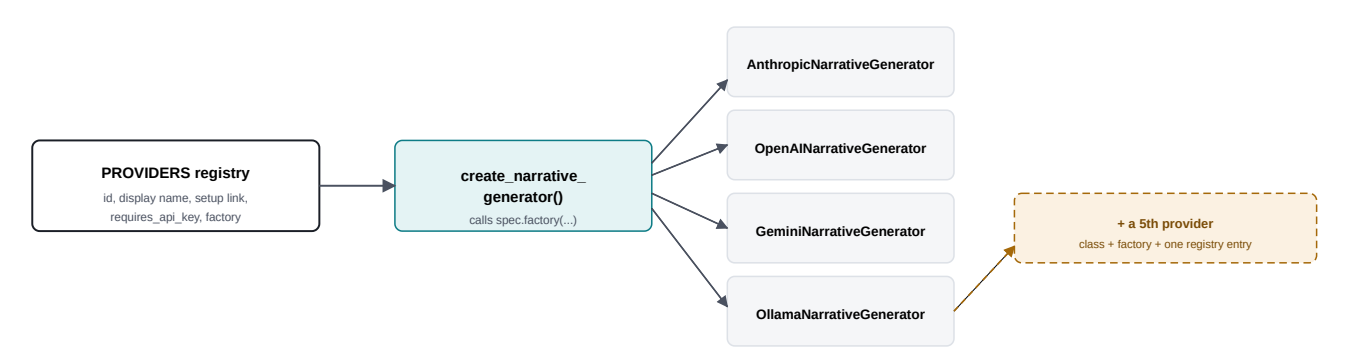


Fig. 3 — Every provider is one registry entry pairing metadata with a factory function; `create_narrative_generator()` never branches on a provider name, so a fifth provider needs no change to the dispatcher itself.

9.1 Extensibility

All four providers implement one `NarrativeGenerator` protocol (a single `generate(prompt) -> str` method) and are declared in one data-driven `PROVIDERS` registry in `core/llm.py`. The CLI's `--provider` choices and the desktop UI's provider dropdown both read that registry directly. Adding a fifth provider — remote or local — requires three additions, all in that one module, with no other file touched: a generator class, a small factory function, and one registry entry.

9.2 Vault storage

The vault stores a provider selection alongside per-provider keys (`llm_provider` , `llm_api_keys` , `llm_model` , `llm_host`), so switching providers doesn't discard a previously-saved key. Vault files written before multi-provider support existed (a bare `llm_api_key` string) are migrated automatically on load.

10 Storage, File Layout & Repository Structure

10.1 A workspace directory (`qv launch --dir <path>`)

```
<workspace>/
├─ Capabilities.md          # skills, grouped under "## Domain" headings
├─ Chronology.md           # roles/timeframes with bullets
├─ Impact.md               # quantified achievements
├─ templates/
│   └─ starter/            # the 7 bundled mission templates
│       └─ community/      # user-contributed .qv-mission.yaml files
├─ bundles/                # qv export output (PDF/DOCX)
├─ .questvector.vault      # encrypted secrets (AES-256-GCM)
└─ .questvector.lock       # advisory cross-process write lock
```

10.2 Repository source layout

```
<repo root>/
├─ README.md
├─ LICENSE
├─ CONTRIBUTING_TEMPLATES.md
├─ pyproject.toml
├─ docs/
│   └─ QuestVector_Spec.pdf # this document
├─ questvector/
│   └─ __init__.py          # __version__ - single source of truth
│   └─ cli.py
│   └─ config.py
│   └─ exceptions.py
│   └─ core/
│       └─ markdown_parser.py
│       └─ matcher.py
│       └─ templates.py
│       └─ exporter.py
│       └─ vault.py
│       └─ workspace.py
│       └─ llm.py
│   └─ data/starter_templates/ # bundled .qv-mission.yaml files
│       └─ desktop/
│           └─ app.py
│           └─ web/          # index.html, app.js, style.css
└─ tests/                  # 142 tests, mirrors the package layout
```

11 Quality, Testing & Packaging

Practice	Detail
Language target	Python 3.12+; every module uses <code>from __future__ import annotations</code> and full PEP 484 type hints
Type checking	Verified clean with <code>mypy</code> against the whole package — zero errors, no suppressions
Linting	Verified clean with <code>pyflakes</code>
Tests	142 <code>pytest</code> tests covering every provider, the vault migration path, the CLI's provider flags, and a dedicated test proving the LLM provider registry is genuinely extensible
Error handling	A single <code>QuestVectorError</code> exception hierarchy (§4); no bare <code>except:</code> anywhere in the codebase
Docstrings	Google-style, on every public module, class, and function
Dependencies	Stdlib-first; third-party only where the standard library has no equivalent (PyYAML, cryptography, reportlab, python-docx, pywebview)
Versioning	<code>questvector.__version__</code> is the single source of truth; <code>pyproject.toml</code> reads it dynamically rather than duplicating the string
Optional extras	<code>pip install questvector[llm-anthropic llm-openai llm-gemini]</code> per provider, or <code>[llm]</code> for all three cloud SDKs at once; Ollama needs no extra (plain HTTP via the standard library)

12 Roadmap — Web Cockpit (Not Yet Built)

NOTHING IN THIS SECTION EXISTS AS CODE

This section documents the original specification's design target and the constraint it must be built under, for planning purposes only.

Planned stack: React/Tailwind UI over a Rust-compiled WebAssembly core, using the browser's File System Access API and IndexedDB for local storage — no server component.

Required distribution model when built, per the same local-only policy as §3: clone the repository, build the Wasm/React bundle locally, and serve the built output from `localhost` — which, unlike a plain `file://` page, qualifies as a secure context for the File System Access API. No public domain will ever host it, and no package will ever be published to the npm registry, matching the Python build's own distribution posture.

The Python build's G-Force matching formula (§5) is this project's own independently-defined algorithm, since the original specification never publishes one. If the Web Cockpit is ever built, its Wasm-based engine is not guaranteed to reproduce identical scores for identical inputs unless it deliberately ports this same algorithm.